Playing BBR with a userspace network stack

Hajime Tazaki

IIJ

April, 2017, Linux netdev 2.1, Montreal, Canada

Linux Kernel Library

- A library of Linux kernel code
- to **be reusable** on various platforms
  - On userspace applications (can be FUSE, NUSE, BUSE)
  - As a core of Unikernel
  - With network simulation (under development)
- Use cases
  - Operating system personality
  - Tiny guest operating system (single process)
  - Testing/Debugging

Motivation

- MegaPipe [OSDI '12]
  - *outperforms baseline Linux .. **582%** (for short connections).*
  - New API for applications (no existing applications benefit)
- mTCP [NSDI '14]
  - *improve... **by a factor of 25** compared to the latest Linux TCP*
  - implement with very limited TCP extensions
- SandStorm [SIGCOMM '14]
  - *our approach ..., **demonstrating 2-10x** improvements*
  - specialized (no existing applications benefit)
- Arrakis [OSDI '14]
  - *improvements of **2-5x in latency and 9x in throughput** .. to Linux*
  - utilize simplified TCP/IP stack (lwip) (loose feature-rich extensions)
- IX [OSDI '14]
  - *improves throughput ... **by 3.6x** and reduces latency **by 2x***
  - utilize simplified TCP/IP stack (lwip) (loose feature-rich extensions)

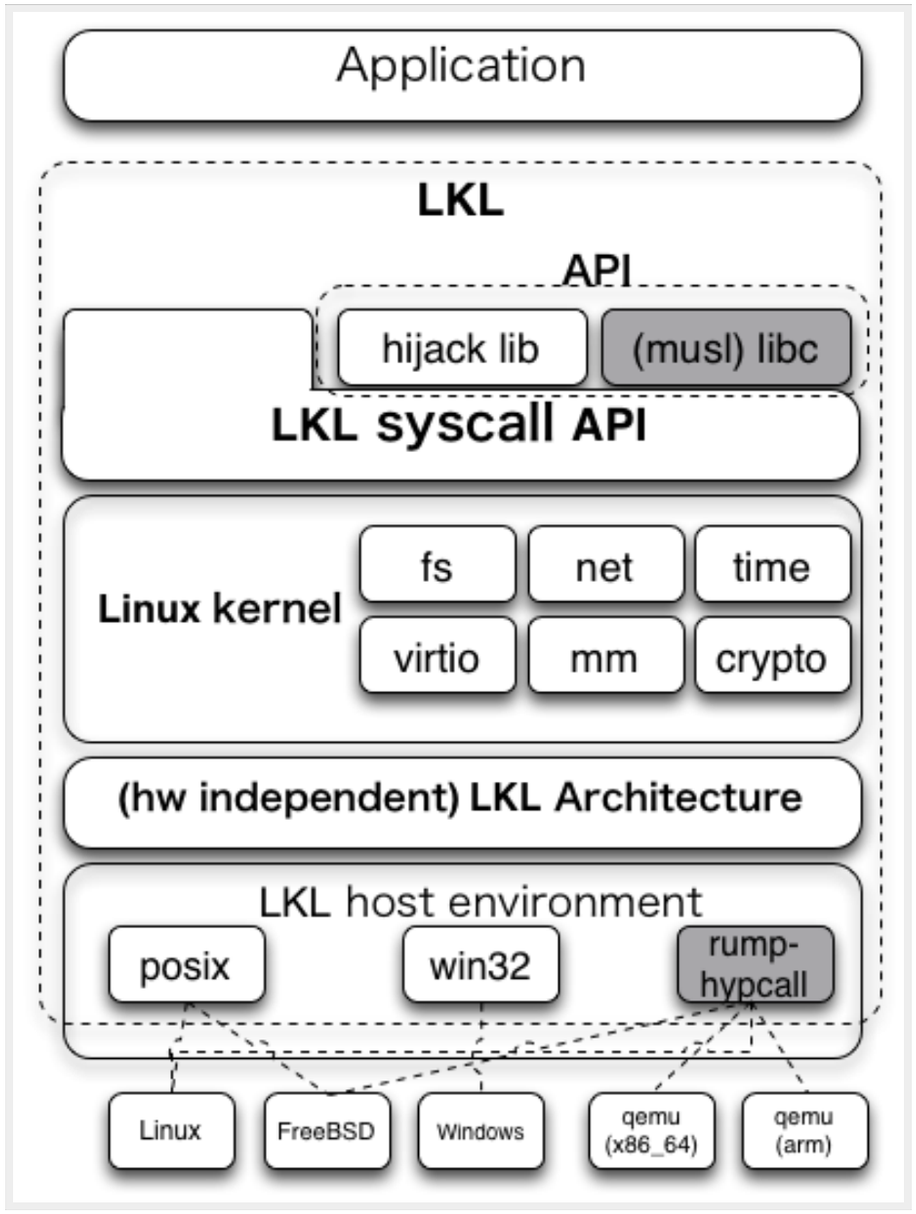
Sigh

Motivation (cont'd)

1. **Reuse** feature-rich network stack, not *re-implement* or *port*
   - re-implement: give up (matured) decades' effort
   - port: hard to track the latest version
2. **Reuse** preserves various semantics
   - syntax level (command line)
   - API level
   - operation level (utility scripts)
3. Reasonable speed with generalized userspace network stack
   - **x1 speed of the original**

LKL outlooks

- h/w independent (**arch/lkl**)
- various platforms
  - Linux userspace
  - Windows userspace
  - FreeBSD user space
  - qemu/kvm (x86, arm) (unikernel)
  - uEFI (EFIDroid)
- existing applications support
  - musl libc bind
  - cross build toolchain



- EFIDroid: http://efidroid.org/

Demo

userspace network stack ?

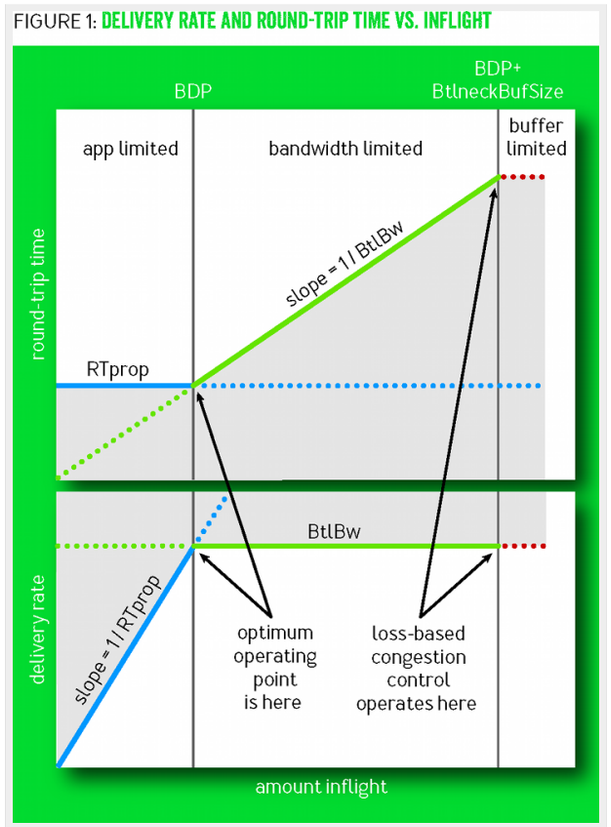
- Concerns about **timing accuracy**
  - how LKL behaves with BBR (requires higher timing accuracy) ?
  - Having network stack in userspace may complicate various optimization

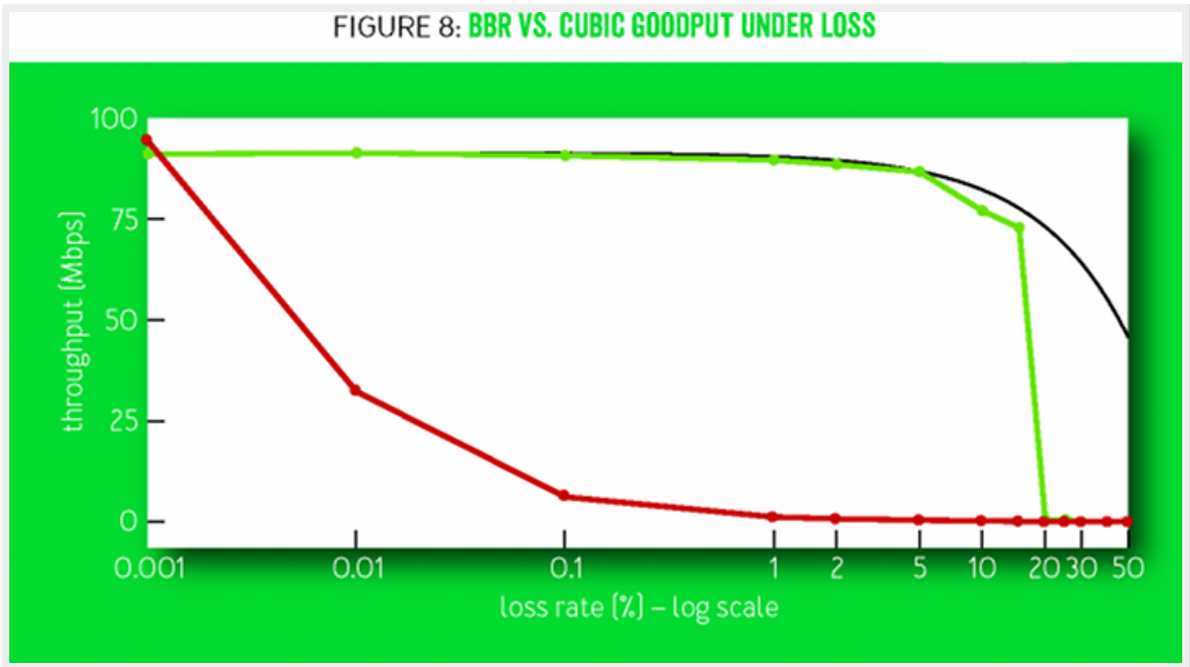
- LKL at netdev1.2
  - https://youtu.be/xP9crHI0aAU?t=34m18s

Playing BBR with LKL

TCP BBR

- **B**ottleneck **B**andwidth and **R**ound-trip propagation time
- Control Tx rate
  - congestion not based on the packet loss
  - estimate MinRTT **and** MaxBW (on each ACK)

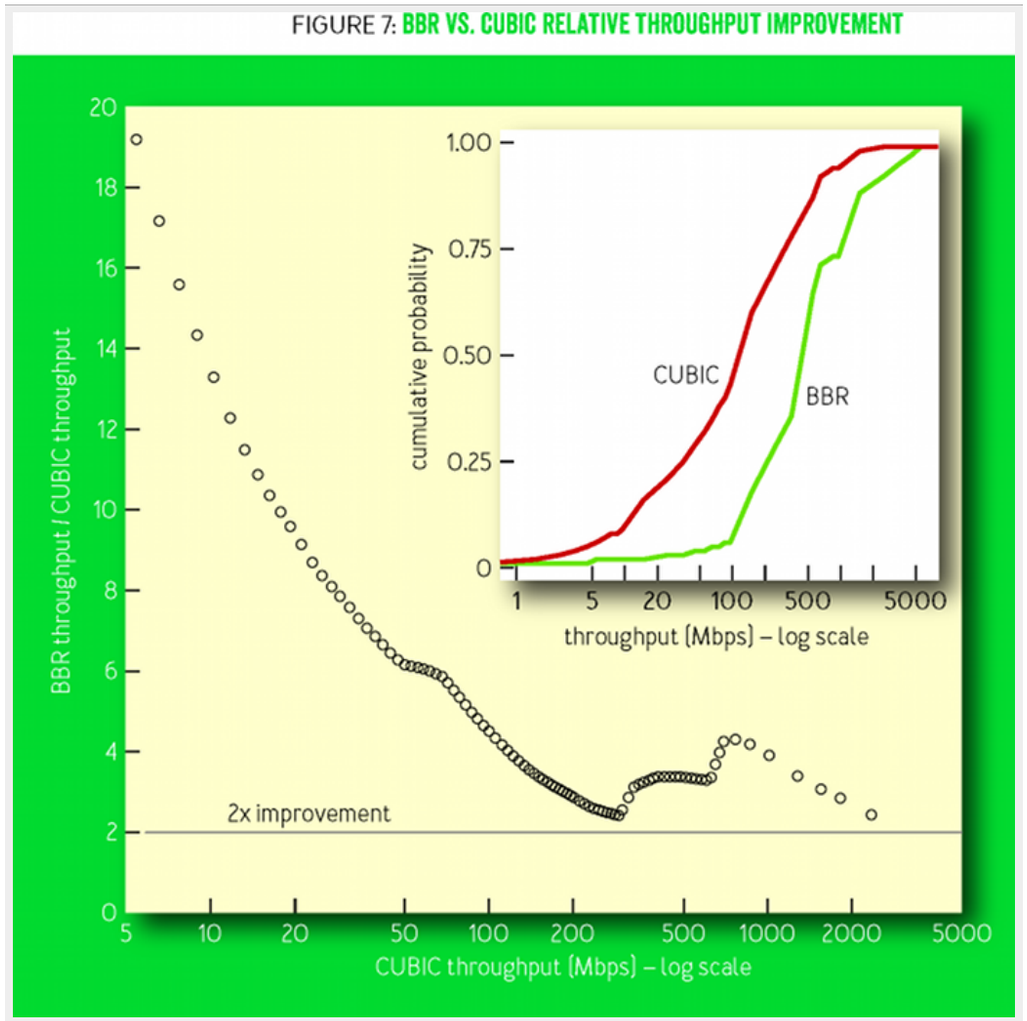




http://queue.acm.org/detail.cfm?id=3022184

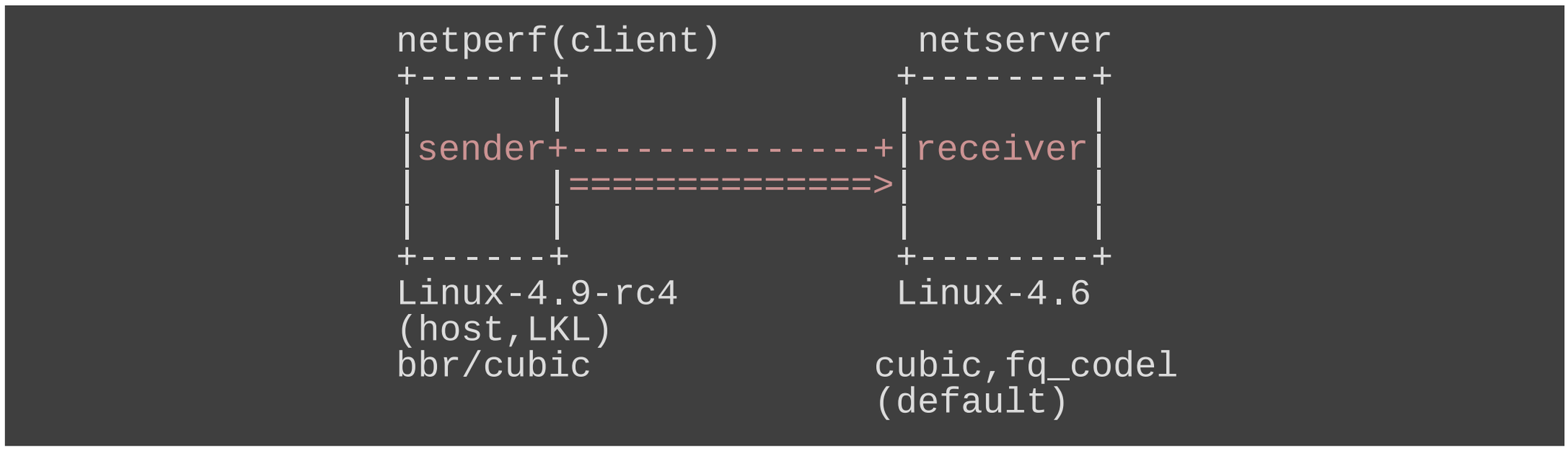
TCP BBR (cont'd)

- On Google's B4 WAN
  (across North America, EU, Asia)
- Migrated from cubic to bbr in 2016
- x2 - x25 improvements

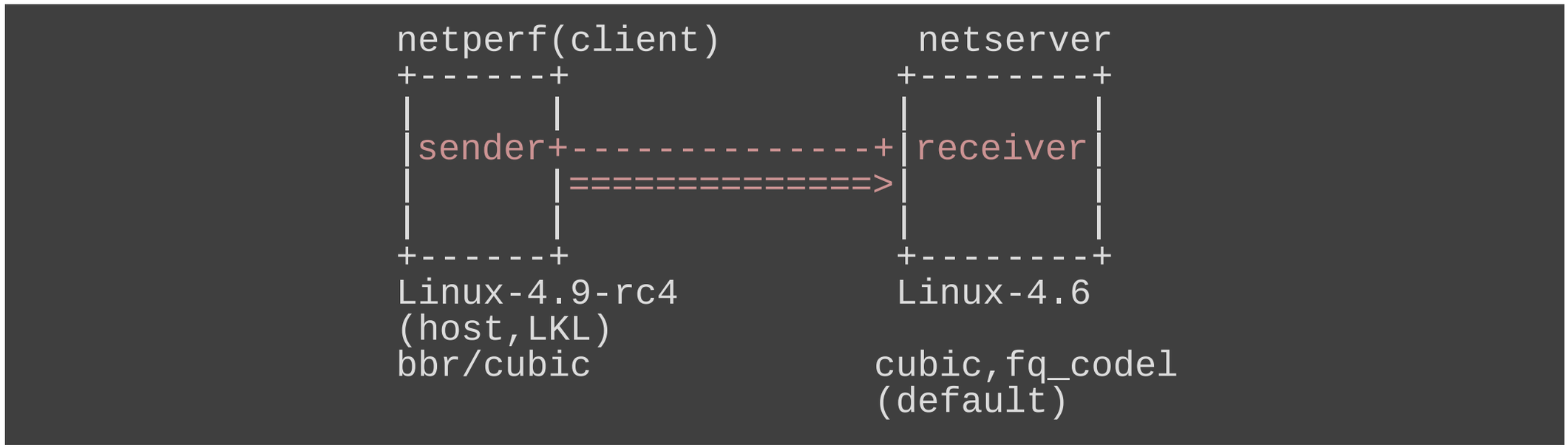


1st Benchmark (Oct. 2016)

- netperf (TCP_STREAM, -K bbr/cubic)
- 2-node 10Gbps b2b link
  - tap+bridge (LKL)
  - direct ixgbe (native)
- No loss, no bottleneck, close link



1st Benchmark



cc	tput (Linux)	tput (LKL)
bbr	9414.40 Mbps	**456.43** Mbps
cubic	9411.46 Mbps	9385.28 Mbps

What ??

- only BBR + LKL shows bad
- Investigation
  - ack timestamp used by RTT measurement needed a precise time event (clock)
  - providing high resolution timestamp improve the BBR performance

Change HZ (tick interval)

cc	tput (Linux,hz1000)	tput (LKL,hz100)	tput (LKL,hz1000)
bbr	9414.40 Mbps	**456.43 Mbps**	**6965.05 Mbps**
cubic	9411.46 Mbps	9385.28 Mbps	9393.35 Mbps

Timestamp on (ack) receipt

From

```
unsigned long long __weak sched_clock(void)
{
    return (unsigned long long)(jiffies - INITIAL_JIFFIES)
                    * (NSEC_PER_SEC / HZ);
}
```

To

```
unsigned long long sched_clock(void)
{
    return lkl_ops->time(); // i.e., clock_gettime()
}
```

cc	tput (Linux)	tput (LKL,hz100)	tput (LKL sched_clock,hz100)
bbr	9414.40 Mbps	456.43 Mbps	**9409.98** Mbps

What happens if no `sched_clock()`?

- low throughput due to longer RTT measurement
- A patch (by Neal Cardwell) to torelate lower in jiffies resolution

```
diff --git a/net/ipv4/tcp_input.c b/net/ipv4/tcp_input.c
index 56fe736..b0f1426 100644
--- a/net/ipv4/tcp_input.c
+++ b/net/ipv4/tcp_input.c
@@ -3196,6 +3196,8 @@ static int tcp_clean_rtx_queue(struct sock *sk, i
                ca_rtt_us = skb_mstamp_us_delta(now, &sack->last_sackt)
        }
        sack->rate->rtt_us = ca_rtt_us; /* RTT of last (S)ACKed packet,
+       if (sack->rate->rtt_us == 0)
+               sack->rate->rtt_us = jiffies_to_usecs(1);
        rtt_update = tcp_ack_update_rtt(sk, flag, seq_rtt_us, sack_rtt_
                                        ca_rtt_us);

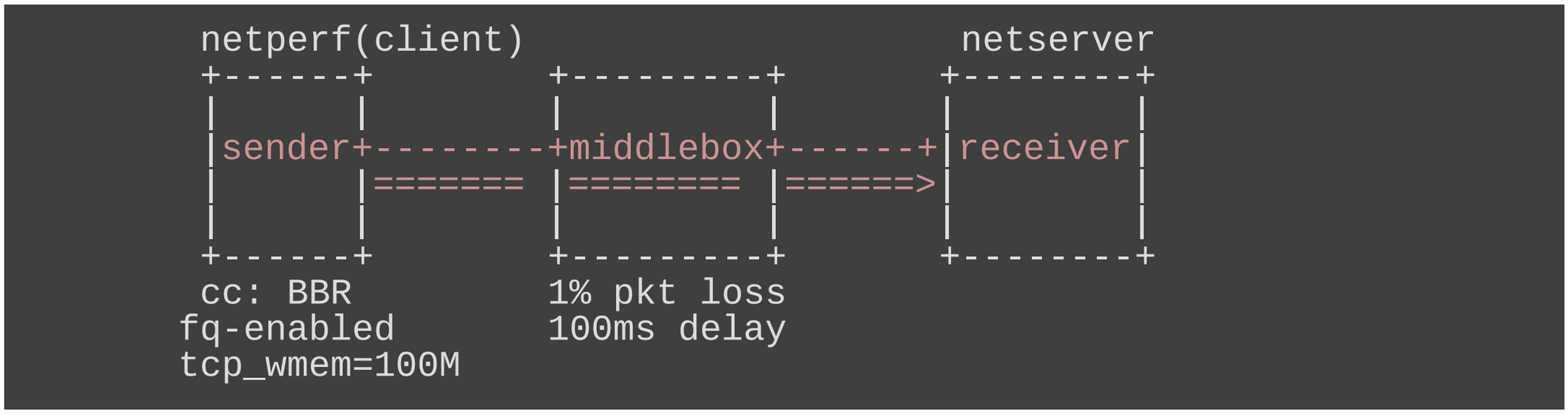
diff --git a/net/ipv4/tcp_rate.c b/net/ipv4/tcp_rate.c
index 9be1581..981c48e 100644
--- a/net/ipv4/tcp_rate.c
+++ b/net/ipv4/tcp_rate.c
```

cc	tput (Linux)	tput (LKL,hz100)	tput (LKL patched,hz100)
bbr	9414.40 Mbps	456.43 Mbps	9413.51 Mbps

- https://groups.google.com/forum/#!topic/bbr-dev/sNwlUuIzzOk

2nd Benchmark

- delayed, lossy network on 10Gbps
  - netem (middlebox)



cc	tput (Linux)	tput (LKL)
bbr	8602.40 Mbps	**145.32 Mbps**
cubic	632.63 Mbps	118.71 Mbps

Memory w/ TCP

- configurable parameter for socket, TCP
  - sysctl -w net.ipv4.tcp_wmem="4096 16384 100000000"
  - delay and loss w/ TCP requires increased buffer

```
"LKL_SYSCTL=net.ipv4.tcp_wmem=4096 16384 100000000"
```

Memory w/ TCP (cont'd)

- default memory size (of LKL): 64MiB
  - the size affects the sndbuf size

```
static bool tcp_should_expand_sndbuf(const struct sock *sk)
{
    (snip)
    /* If we are under global TCP memory pressure, do not expand.  */
    if (tcp_under_memory_pressure(sk))
        return false;
    (snip)
}
```

Timer relates

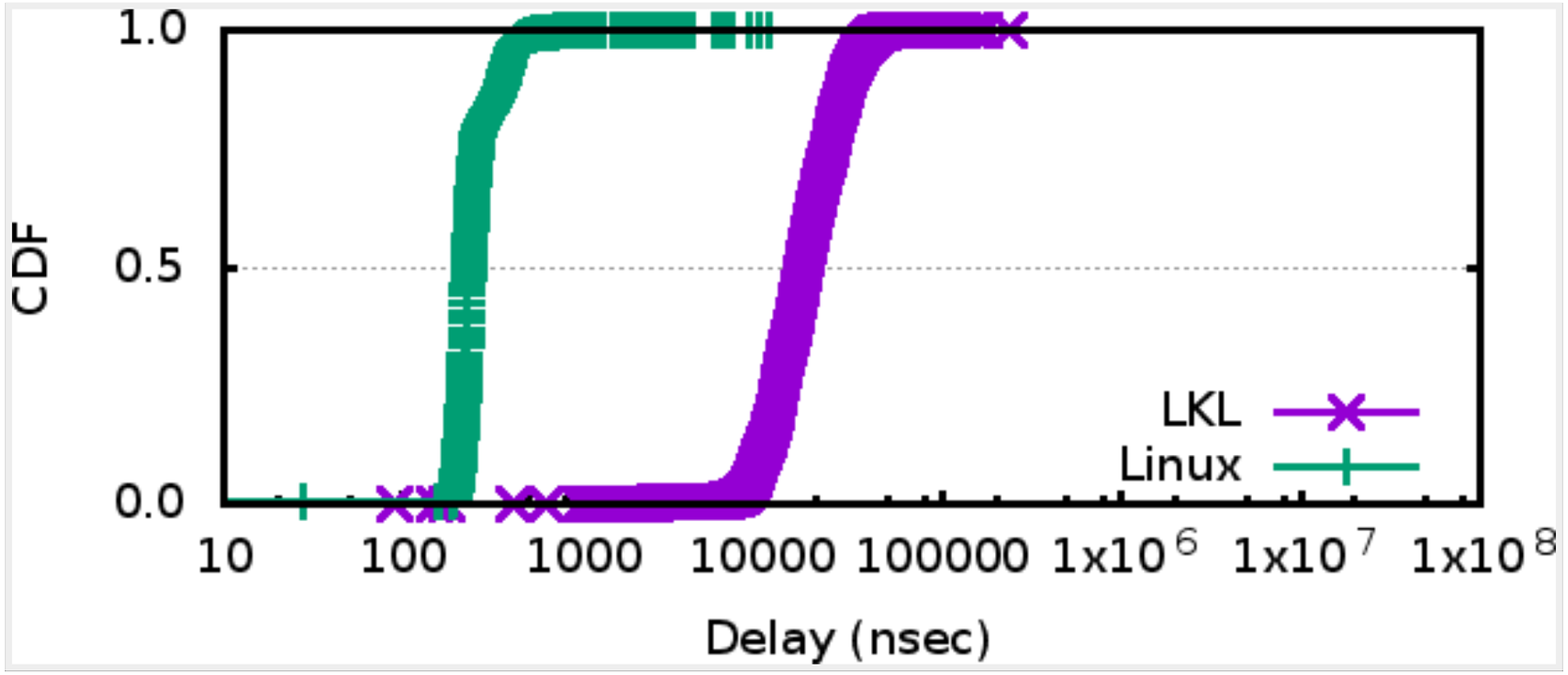
- CONFIG_HIGH_RES_TIMERS enabled
  - fq scheduler uses
  - properly transmit packets with probed BW
- fq configuration
  - instead of `tc qdisc add fq`

fq scheduler

- Every fq_flow entry scheduled schedule a timer event
  - with high-resolution timer (in nsec)

```
static struct sk_buff *fq_dequeue()
 => void qdisc_watchdog_schedule_ns()
  => hrtimer_start()
```

How slow high-resolution timer ?



Delay = (**nsec of expiration**) - (**nsec of scheduled**)

Scheduler improvement

- LKL's scheduler
  - outsourced based on thread impls (green/native)
- minimum delay of timer interrupt (of LKL emulated)
  - >60 usec (green thread)
  - >20 usec (native thread)

Scheduler improvement

1. avoid system call (clock_nanosleep) when block
   - busy poll (watch clock instead) if sleep is < 10usec
   - 60 usec => 20 usec

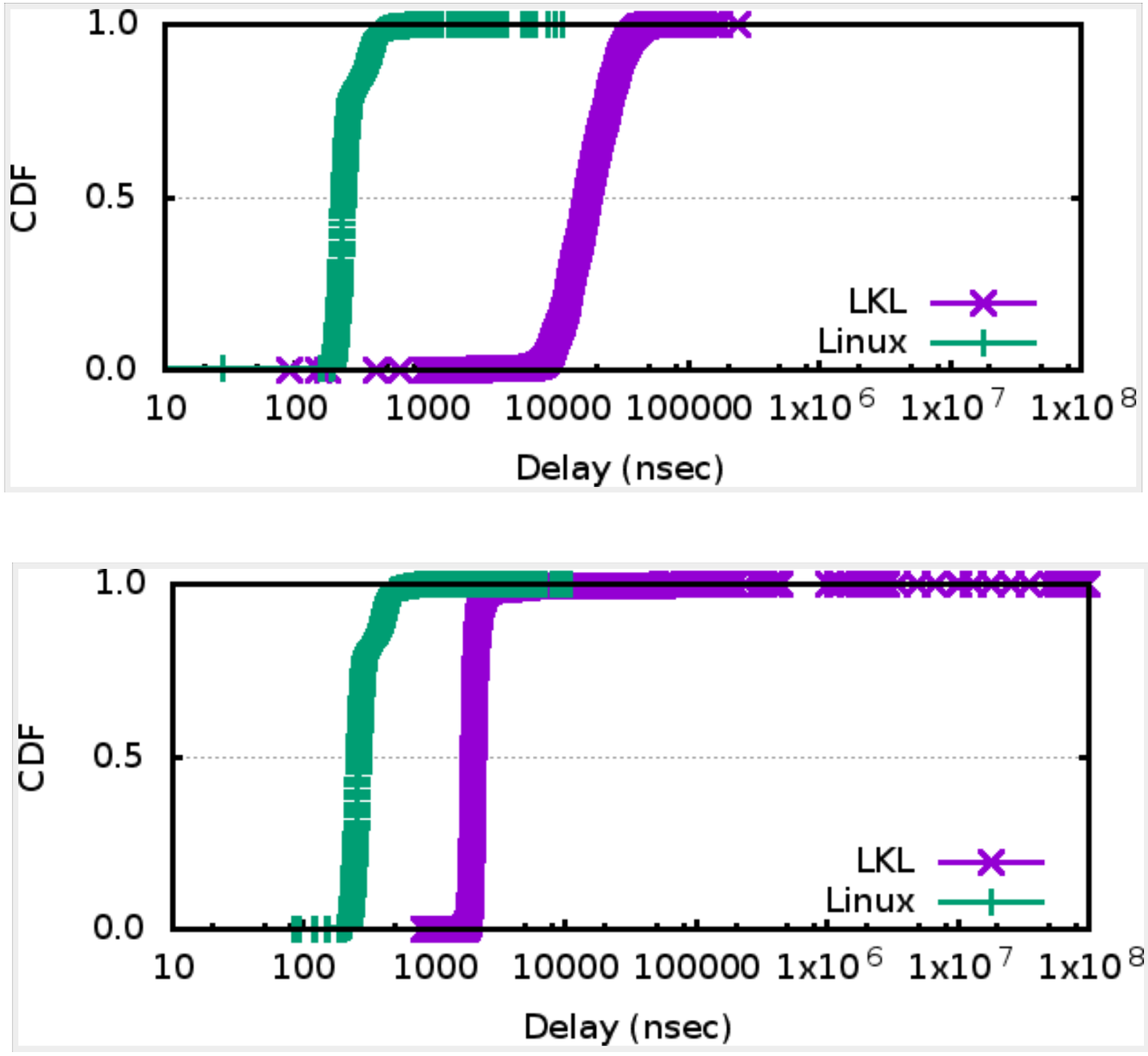
```
int sleep(u64 nsec) {
   /* fast path */
   while (1) {
            if (nsec < 10*1000) {
               clock_gettime(CLOCK_MONOTONIC, &now);
               if (now - start > nsec)
                  return;
            }
   }

   /* slow path */
   return syscall(SYS_clock_nanosleep)
}
```

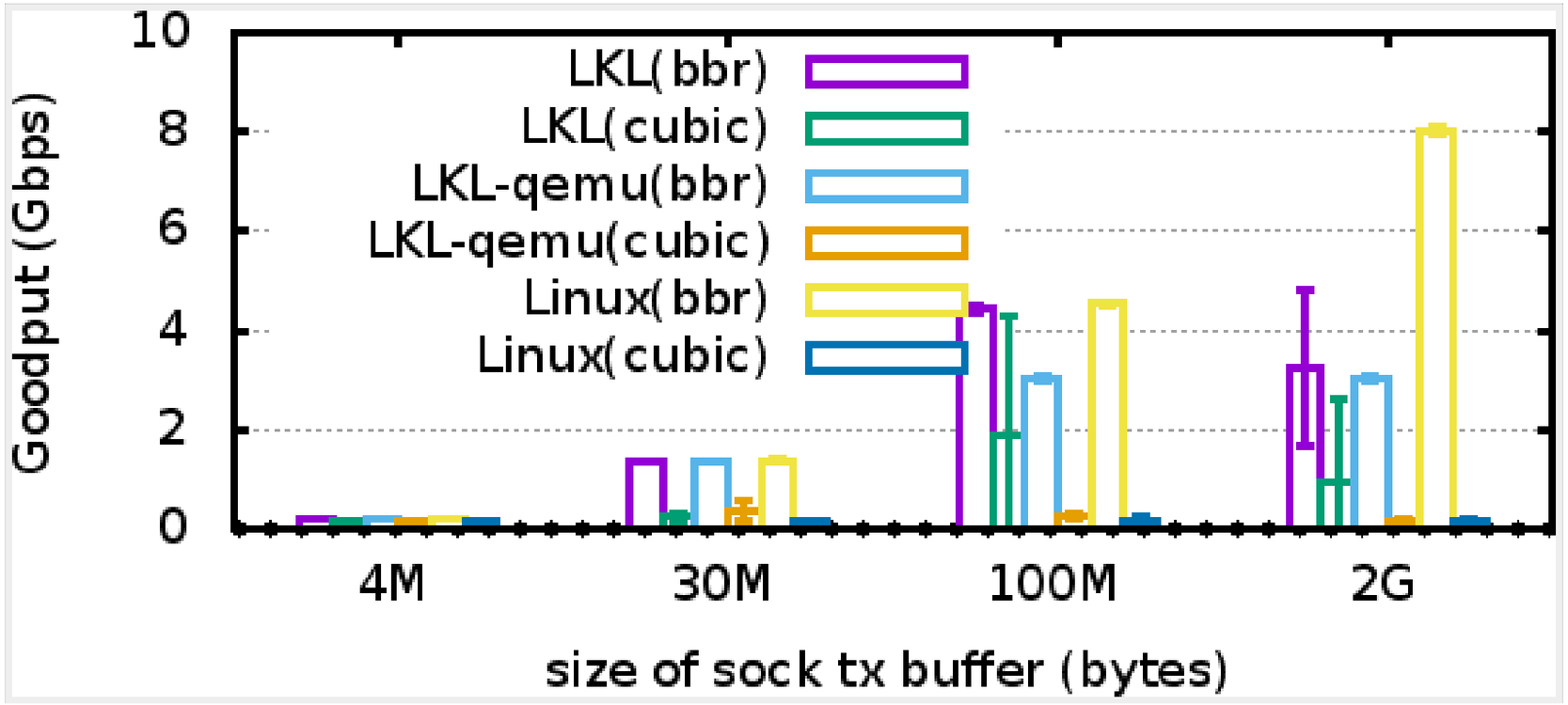
2. reuse green thread stack (avoid mmap per a timer irq)
   - 20 usec => 3 usec

Timer delay improved ?

- Before (top), After (bottom)



Results (TCP_STREAM, bbr/cubic)




```
 netperf(client)                      netserver
 +------+         +---------+        +--------+
 |      |         |         |        |        |
 |sender+---------+middlebox+-------+|receiver|
 |      |======== |========= |======>|        |
 |      |         |         |        |        |
 +------+         +---------+        +--------+
 cc: BBR          1% pkt loss
fq-enabled        100ms delay
tcp_wmem=100M
```

Patched LKL

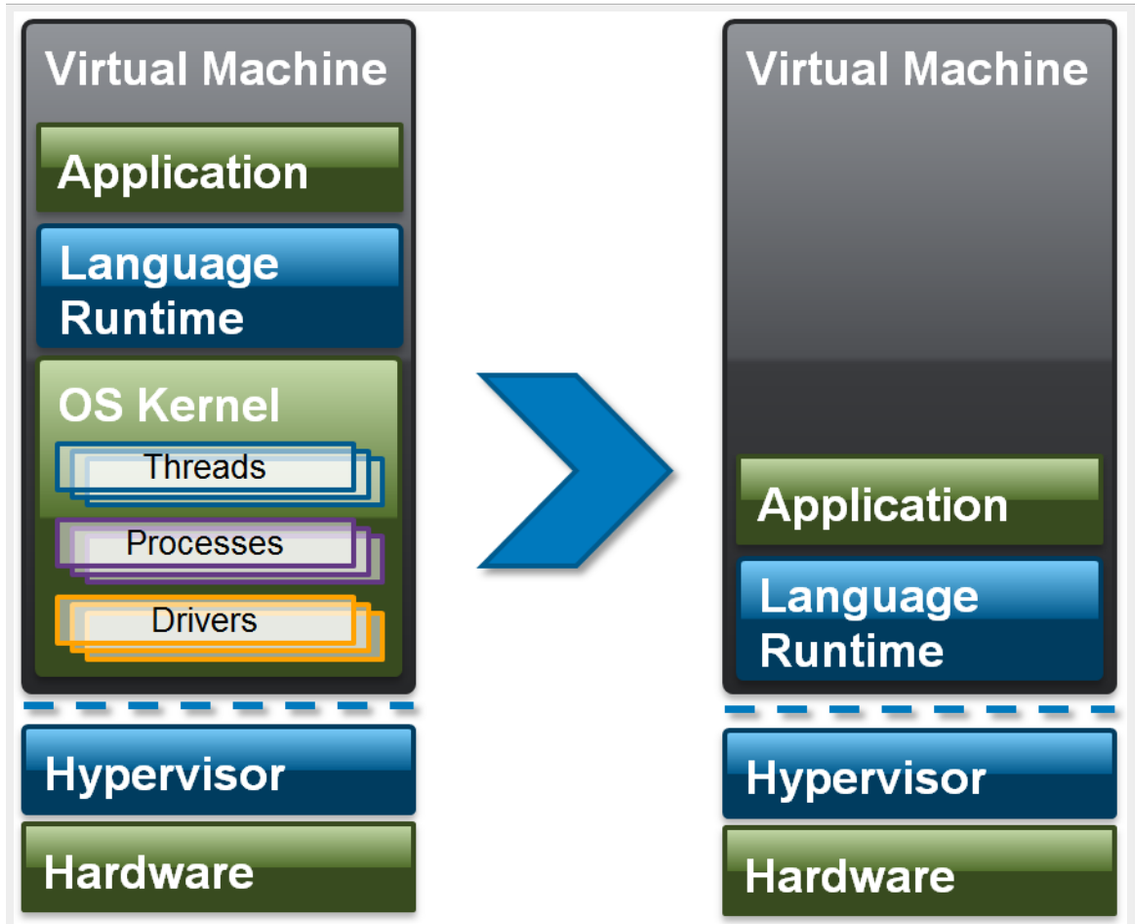
1. add **sched_clock()**
2. add **sysctl** configuration i/f (net.ipv4.tcp_wmem)
3. make system **memory** configurable (net.ipv4.tcp_mem)
4. enable **CONFIG_HIGH_RES_TIMERS**
5. add **sch-fq** configuration
6. scheduler hacked (*uspace specific*)
   - avoid syscall for short sleep
   - avoid memory allocation for each (thread) stack
7. (TSO, csum offload, by Jerry/Yuan from Google, netdev1.2)

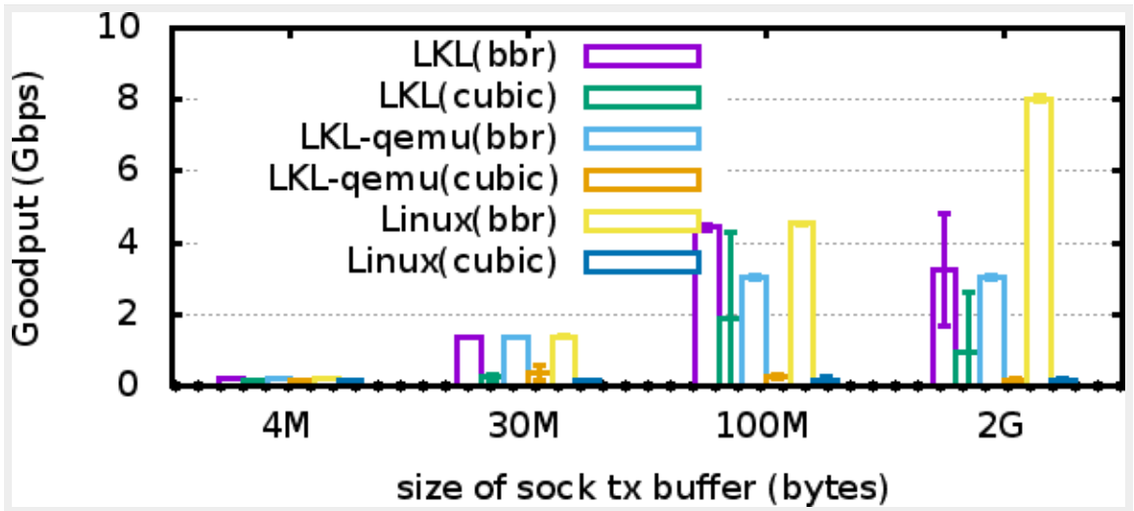
Next possible steps

- `do` profile `while` (lower LKL performance)
  - e.g., context switch of uspace threads
- Various short-cuts
  - busy polling I/Os (packet, clock, etc)
  - replacing packet I/O (packet_mmap)
- short packet performance (i.e., 64B)
- practical workload (e.g., HTTP)
- > 10Gbps link

on qemu/kvm ?

- Based on rumprun unikernel
- Performance under investigation
- No scheduler issue (not depending on syscall)





- http://www.linux.com/news/enterprise/cloud-computing/751156-are-cloud-operating-systems-the-next-big-thing-

Summary

- Timing accuracy concern was right
- performance obstacle in userspace execution
  - scheduler related
  - alleviated somehow
- Timing severe features degraded from Linux
- other options (unikernel)
  - The benefit of **reusable code**

References

- LKL
  - https://github.com/lkl/linux
- Other related repos
  - https://github.com/libos-nuse/lkl-linux
  - https://github.com/libos-nuse/frankenlibc
  - https://github.com/libos-nuse/rumprun

Backup

Alternatives

- Full Virtualization
  - KVM
- Para Virtualization
  - Xen
  - UML
- Lightweight Virtualization
  - Container/namespaces

What is ***not*** LKL ?

- **not** specific to a userspace network stack
- Is a **reusable** library that we can use everywhere (in theory)

How others think about userspace ?

DPDK is not Linux (@ netdev 1.2)

- The model of DPDK isn't compatible with Linux
  - break security model (protection never works)
- XDP is Linux

userspace network stack (checklist)

- Performance
- Safety
- Typos take the entire system down
- Developer pervasiveness
- Kernel reboot is disruptive
- Traffic loss

ref: XDP Inside and Out
(https://github.com/iovisor/bpf-docs/blob/master/XDP_Inside_and_Out.pdf)

TCP BBR (cont'd)

BBR requires

- packet pacing
- precise RTT measurement

```
function onAck(packet)
  rtt = now - packet.sendtime
  update_min_filter(RTpropFilter, rtt)
  delivered += packet.size
  delivered_time = now
  deliveryRate = (delivered - packet.delivered) /
          (delivered_time - packet.delivered_time)
  if (deliveryRate > BtlBwFilter.currentMax ||
           ! packet.app_limited)
     update_max_filter(BtlBwFilter, deliveryRate)
  if (app_limited_until > 0)
     app_limited_until = app_limited_until - packet.size
```

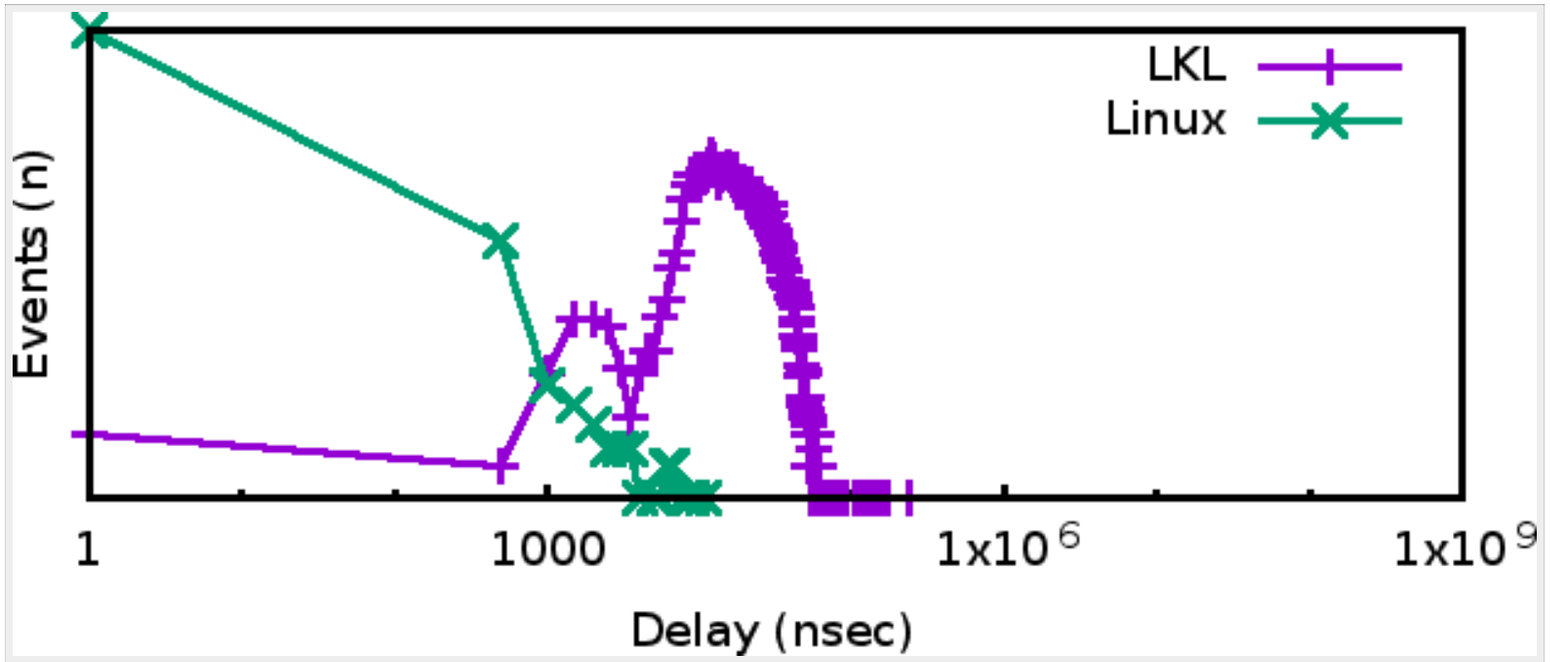
http://queue.acm.org/detail.cfm?id=3022184

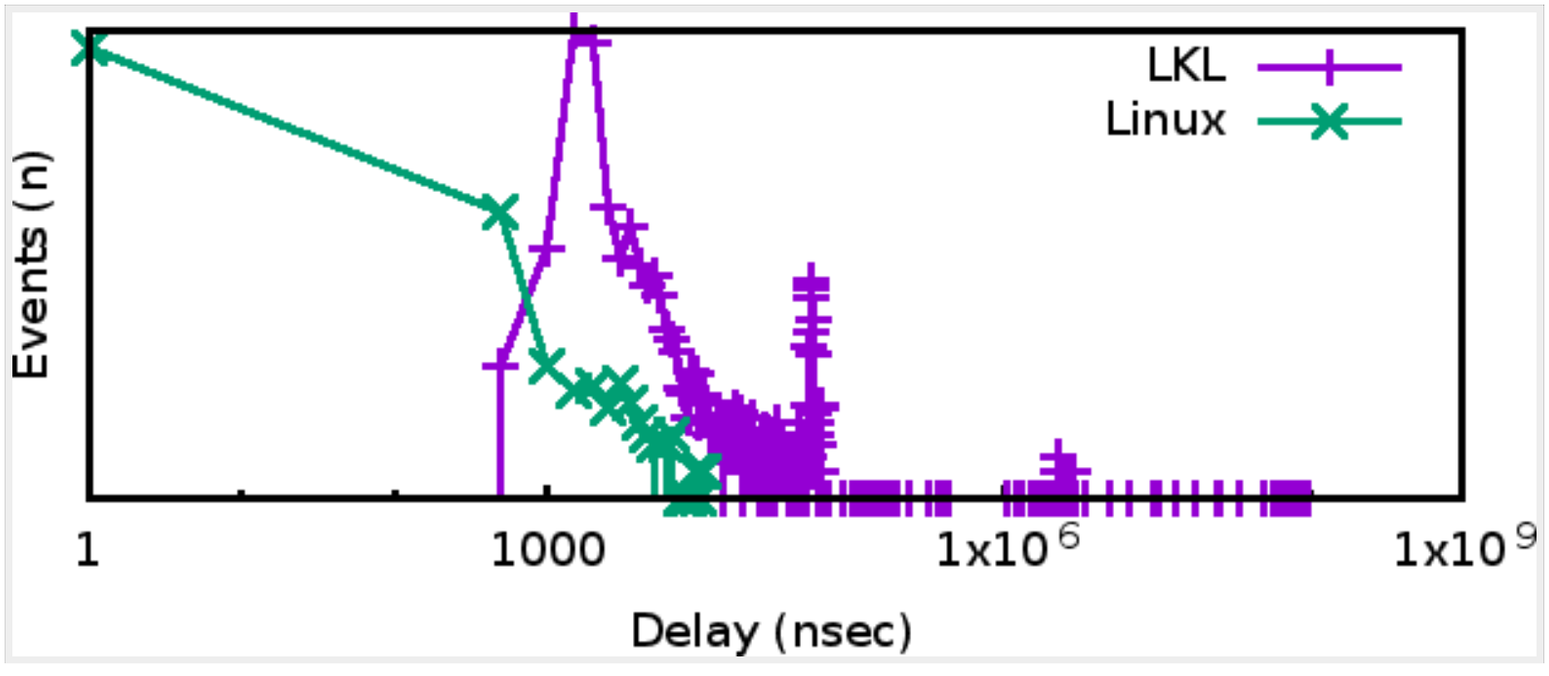
How timer works ?

1. schedule an event
2. add (hr)timer list queue (hrtimer_start())
3. (check expired timers in timer irq) (hrtimer_interrupt())
4. invoke callbacks (__run_hrtimer())

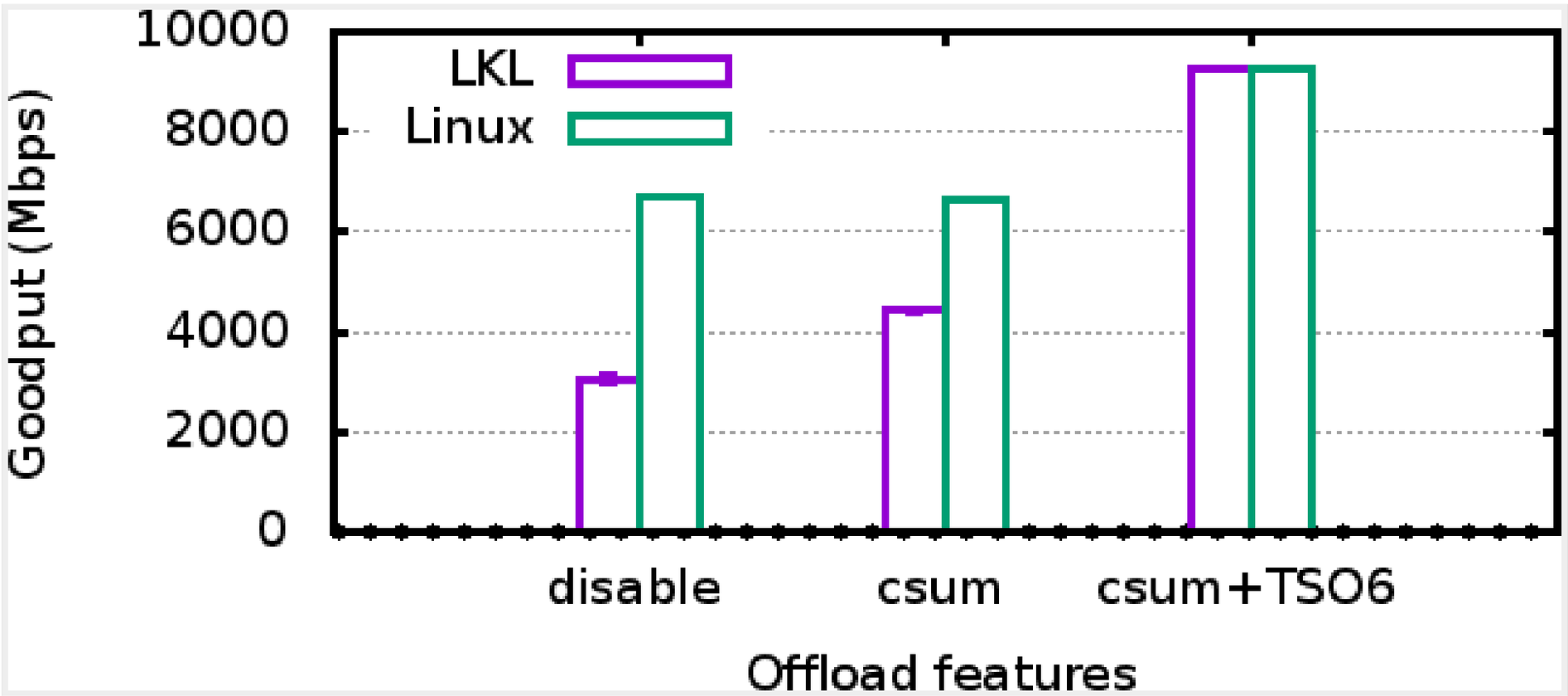
Timer delay improved ?

- Before (top), After (bottom)





IPv6 ready


10000
8000
6000
4000
2000
0
Goodput (Mbps)
LKL
Linux
disable
csum
csum+TSO6
Offload features

how timer interrupt works ?

native thread ver.

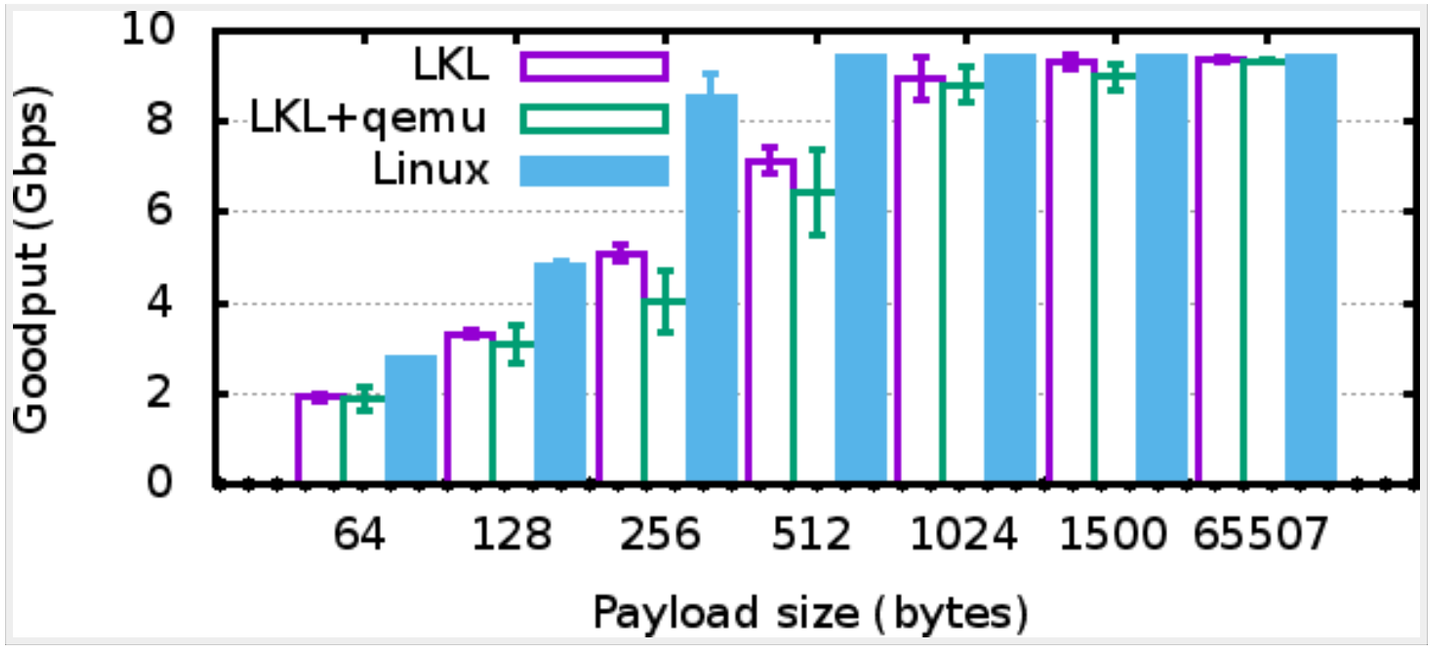
1. timer_settime(2)
   - instantiate a pthread
2. wakeup the thread
3. trigger a timer interrupt (of LKL)
   - update jiffies, invoke handlers

green thread ver.

1. instantiate a green thread
   - malloc/mmap, add to sched queue
2. schedule an event
   - clock_nanosleep(2) until next event)
   - or do something (goto above)
3. trigger a timer interrupt

TSO/Checksum offload

- virtio based
- guest-side: use Linux driver



- TCP_STREAM (cubic, no delay)